INTELLIGENT ALERT CLASSIFICATION FOR IOT-BASED ELDER CARE FALL RESPONSE SYSTEMS

INDIVIDUAL THESIS

[Student Full Name]

([Student ID])

Individual component: Alert Classification Module (Component 4 of AegisCare)

Group ID: [Group ID]

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science Honours in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

September 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis. Generative AI tools were used to assist with drafting, code debugging, and document formatting during this project; all technical results, model training, and evaluation figures presented in this thesis were independently verified by the candidate. A full disclosure is provided in Appendix B.

Student name: [Full name] Student ID: [Student ID]

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: [Name]		
Co-supervisor: [Name]		

ABSTRACT

Falls among older adults living independently are rarely explained by a single cause, yet most Internet of Things (IoT)-based elder-care systems stop at detecting that a fall occurred without deciding who should respond and how urgently. This individual component addresses that gap within the AegisCare elder-care monitoring system by designing, implementing, and evaluating an Alert Classification module that routes every confirmed fall to one of three response tiers: self-guided recovery for the elder, notification to a caregiver, or escalation to an emergency contact. The component combines nine features drawn from three sources — the fall event itself, environmental context, and the elder's stored risk profile — into a single decision, computed by a gradient-boosted tree classifier (XGBoost) trained on a labelled dataset of 50,000 samples. Following systematic hyperparameter tuning using randomized search with three-fold cross-validation, the tuned model achieved 78.47% test accuracy with a train-test generalization gap of 1.06 percentage points, reduced from 2.72 points in an untuned baseline. Analysis of the model's behaviour on confirmed falls in the training data showed that 67.3% of cases warranted an emergency-tier response, 31.1% a caregiver-tier response, and only 1.6% qualified for self-recovery guidance, indicating that most physically confirmed falls require human intervention. The component's design was revised twice during development to remove logical inconsistencies between independently evaluated rules and the classifier's own decision, and two deployment-level engineering issues — a library version incompatibility and a categorical feature-encoding mismatch — were identified, diagnosed, and resolved, with both documented as reusable lessons for similar systems. The findings demonstrate that a model-driven, single-authority routing decision is both technically achievable and more logically consistent than a system that layers independent rules on top of a classifier's output, while also surfacing the evaluation-methodology discipline required to report such a system's performance honestly.

Keywords: alert classification, elder care, fall detection, gradient boosting, Internet of Things, multi-class classification, XGBoost

ACKNOWLEDGEMENTS

I would like to thank my supervisor and co-supervisor for their guidance throughout the design, implementation, and evaluation of this component, and my fellow team members for their collaboration in integrating this module into the complete AegisCare system. I am also grateful to the Faculty of Computing at the Sri Lanka Institute of Information Technology for the resources and support provided throughout this project.

TABLE OF CONTENTS

(Right-click and choose "Update Field", or press Ctrl+A then F9 in Word, to populate this table of contents before submission.)

1 INTRODUCTION 8
1.1 Overall project context 8
1.2 Individual component context 8
1.3 Problem statement 8
1.4 Research gap 9
1.5 Aim and objectives 9
1.6 Research questions or hypotheses 9
1.7 Scope limitations and assumptions 10
1.8 Individual contribution 10
1.9 Thesis structure 10
2 LITERATURE REVIEW AND RELATED WORK 10
2.1 Review method and scope 10
2.2 Domain and theoretical foundations 11
2.3 Related research 11
2.4 Existing systems products and techniques 12
2.5 Gap synthesis and conceptual framework 12
3 RESEARCH METHODOLOGY 13
3.1 Research design 13
3.2 Data participants or study materials 13
3.3 Experimental or study procedure 13
3.4 Measures and evaluation metrics 13
3.5 Data analysis 14
3.6 Ethics privacy security and responsible AI 14
3.7 Reproducibility and research artefacts 14
4 PRODUCT AND SOLUTION DESIGN 14
4.1 Requirements 14
4.2 Component architecture within the complete solution 15
4.3 Design alternatives and decisions 16
4.4 Detailed design 16
4.5 Standards and professional practice 17
5 IMPLEMENTATION INTEGRATION AND TESTING 18
5.1 Development environment and technology stack 18
5.2 Component implementation 18

5.3 Integration with the other components 19
5.4 Verification and software testing 19
5.4.1 Test case summary 20
5.5 Deployment and operation 21
6 RESULTS AND VALIDATION 22
6.1 Research results 22
6.2 Product and technical results 23
6.3 User or stakeholder validation 23
6.4 Integration results 24
6.5 Summary of findings 24
7 DISCUSSION 24
7.1 Interpretation of findings 24
7.2 Comparison with literature baselines and existing solutions 25
7.3 Research contribution 25
7.4 Product contribution 25
7.5 Limitations and threats to validity 25
7.6 Implications 26
8 COMMERCIALISATION AND SUSTAINABILITY 26
8.1 Value proposition and target users 26
8.2 Market and competitive position 26
8.3 Business or sustainability model 27
8.4 Scalability intellectual property and compliance 27
8.5 Contribution to the Sustainable Development Goals 27
9 CONCLUSION AND FUTURE DIRECTIONS 28
9.1 Conclusion 28
9.2 Achievement of objectives 28
9.3 Individual research and product contributions 28
9.4 Future directions 29
REFERENCES 29
GLOSSARY 30
APPENDICES 30
Appendix A Research instruments or extended results 30
Appendix B Academic integrity and AI use disclosure 30

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
DSR	Design Science Research
IoT	Internet of Things
LOSO	Leave-One-Subject-Out (cross-validation)
MAE	Mean Absolute Error
MQTT	Message Queuing Telemetry Transport
PERS	Personal Emergency Response System
PPG	Photoplethysmography
REST	Representational State Transfer
SDG	Sustainable Development Goal
TLS	Transport Layer Security
XGBoost	Extreme Gradient Boosting

1 INTRODUCTION

1.1 Overall project context

Falls among older adults living independently are one of the leading causes of injury, hospitalization, and loss of independence worldwide, and their causes are rarely one-dimensional. A fall may result from a moment of physical instability, from a hazardous environment such as a poorly lit or wet floor, from an underlying cardiovascular event such as a sudden drop in blood pressure, or from an emergency response that arrives too late or is directed at the wrong person. AegisCare is a multimodal elder-care monitoring system, developed by a team of four students, that addresses these contributing factors through four cooperating components: a wearable Context-Aware Activity Recognition module that detects falls and classifies movement from inertial sensor data; an Environmental Risk Mapping module that continuously scores ambient hazards from a network of Internet of Things (IoT) sensors; a Physiological Risk Monitoring module that tracks cardiovascular indicators from a low-cost optical sensor; and an Alert Classification module that decides, once a fall is confirmed, who should be notified and how urgently.

This individual thesis documents the fourth of these four components. Where the other three components each sense a different dimension of fall risk, the Alert Classification module occupies the position of decision layer: it does not sense a new modality, but instead consumes the outputs of the fall-detection component together with environmental context and the elder's stored risk profile, and produces the system's actionable output.

1.2 Individual component context

The Alert Classification module sits downstream of the wearable fall-detection component. Whenever that component confirms a physical fall, the Alert Classification module is invoked with a structured description of the event, and returns a routing decision naming one of three response tiers: Voice Alert, in which the elder's own device delivers spoken recovery guidance; Caregiver Alert, in which the registered caregiver is notified; or Emergency Alert, in which a designated emergency contact is notified in real time. This decision is computed by a trained machine-learning classifier rather than a fixed rule table, so that the routing behaviour can reflect the interaction between a fall's physical characteristics, the environment in which it occurred, and the individual elder's personal risk factors.

1.3 Problem statement

A fall-detection signal alone answers only one question: did a fall occur? It does not address a second, equally important question: given that a fall occurred, who should respond, and how urgently? Treating every detected fall identically is both inefficient and potentially unsafe. A mild stumble that the elder can recover from independently does not require the same response as a severe fall that leaves the elder unresponsive. Routing every event to a caregiver risks alert fatigue and delayed response to genuine emergencies, while routing every event to an emergency contact is neither necessary nor sustainable at scale. The problem addressed by this component is therefore not fall detection itself, but the comparatively under-studied problem

of deciding, once a fall has been detected, which of several possible recipients should be notified, and with what urgency.

1.4 Research gap

A review of the fall-detection literature, presented in full in Chapter 2, shows that most published systems stop at the binary detection problem — distinguishing a fall from normal activity — without addressing downstream response routing. Where response tiers are considered, commercial systems typically rely on a human call-centre operator to triage the event, rather than an automated classifier, and academic systems that do propose automated severity classification rarely report an evaluation methodology rigorous enough to distinguish genuine model performance from artefacts of the evaluation protocol itself. This component addresses both gaps: it proposes a fully automated, model-driven routing decision, and it reports that decision's evaluation with explicit attention to methodological soundness, including a documented instance of correcting an initially over-optimistic evaluation result.

1.5 Aim and objectives

The aim of this component is to design, implement, and evaluate a model-driven alert classification module that routes every confirmed fall event, within the AegisCare elder-care system, to the appropriate response tier with measurable, honestly reported accuracy. The specific objectives are:

- To design a feature representation that combines fall characteristics, environmental context, and elder-specific risk factors into a single classification input.
- To train and systematically tune a multi-class classifier that outputs a probability distribution over three response tiers, rather than a single hard-coded rule.
- To evaluate the trained classifier using a held-out test set and to report per-class precision, recall, and F1-score, alongside overall accuracy and the train-test generalization gap.
- To integrate the classifier into the wider AegisCare system so that its decision is delivered to the correct recipient application in real time.
- To identify, diagnose, and document any deployment-level engineering issues encountered during integration, as a contribution to reproducible practice for similar systems.

1.6 Research questions or hypotheses

This component investigates the following research questions.

- RQ1: Can a nine-feature representation combining fall event data, environmental context, and elder risk profile support multi-class alert routing with accuracy comparable to reported binary fall-detection benchmarks, given the added difficulty of distinguishing three response tiers rather than two classes?

- RQ2: Does systematic hyperparameter tuning improve the trained classifier's generalization to unseen data, as measured by the gap between training and test accuracy, without a corresponding loss in test accuracy itself?
- RQ3: Does gating the elder-facing recovery-guidance feature directly on the classifier's own decision, rather than on an independently evaluated risk-level rule, eliminate the possibility of the two subsystems disagreeing about who should respond to a given fall?

1.7 Scope limitations and assumptions

This component is scoped to the alert-routing decision itself; it does not perform fall detection, environmental sensing, or physiological monitoring, which are the subjects of the other three components of AegisCare and are described only insofar as they provide input to this component. The classifier was trained on a labelled synthetic dataset of 50,000 samples constructed to reflect plausible combinations of fall severity, context, and elder risk factors, rather than a corpus of real, clinically verified fall events; this is a limitation discussed in detail in Chapter 7. It is assumed that the upstream fall-detection component correctly identifies confirmed falls before this component is invoked, and that the elder's stored risk profile (age, prior fall count, assistive-device use) is accurate and up to date.

1.8 Individual contribution

The candidate independently designed the nine-feature input representation, implemented the classification pipeline using a scikit-learn preprocessing stage and an XGBoost classifier, conducted the hyperparameter search and evaluation, diagnosed and resolved two deployment-level engineering issues encountered during integration with the wider AegisCare system, and authored this thesis. The candidate collaborated with the other three team members on defining the shared MQTT communication contract and the elder risk-profile schema consumed by this component, documented in the team's Common Report.

1.9 Thesis structure

The remainder of this thesis is organized as follows. Chapter 2 reviews related work in fall detection, ensemble learning, and automated alert routing. Chapter 3 describes the research methodology, including the Design Science Research approach adopted and the dataset used. Chapter 4 presents the product and solution design, including the two-branch architecture and the feature representation. Chapter 5 describes the implementation, integration with the other three components, and testing. Chapter 6 reports the evaluation results. Chapter 7 discusses the findings, their interpretation, and the limitations of the work. Chapter 8 considers the commercialisation and sustainability of the component. Chapter 9 concludes the thesis and outlines future directions.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

This review covers four bodies of literature relevant to the Alert Classification component: fall detection and severity assessment for older adults; ensemble tree-based machine learning methods suitable for structured, tabular classification problems; commercial and academic approaches to post-detection response routing; and evaluation-methodology practices for reporting classifier performance honestly. Sources were identified through targeted searches of IEEE Xplore, ACM Digital Library, and general web search for peer-reviewed articles, conference proceedings, and vendor documentation, prioritizing sources that specifically addressed response routing or severity classification rather than binary fall detection alone, since the latter is comparatively well-covered but does not directly address this component's research questions.

2.2 Domain and theoretical foundations

Falls are recognized as a leading cause of injury-related morbidity and mortality among older adults, and the risk of both a fall occurring and a fall having severe consequences increases with age, prior fall history, and reduced mobility [1]. This clinical understanding motivates the inclusion of age, prior fall count, and assistive-device use as input features in the classification task addressed by this component: identical physical fall characteristics carry different clinical significance depending on the individual's risk profile. Ambient assisted living research has established that combining multiple sensing modalities — wearable, ambient, and increasingly physiological — provides a more complete picture of fall risk than any single modality alone [1], which is the architectural principle underlying the four-component structure of the wider AegisCare system, of which this component is the response-routing layer.

2.3 Related research

Fall detection for older adults using inertial and IoT-connected sensors has been studied extensively. Karar et al. surveyed IoT-based fall detection systems and categorized detection approaches into rule-based and machine-learning-based methods, finding that most published systems focus on the binary detection problem — distinguishing a fall from normal activity — using accelerometer and gyroscope data from wearable devices, without addressing downstream response routing [1]. This finding directly motivated the research gap this component addresses: detecting a fall is treated in the literature as the end of the problem, when in a deployed system it is only the beginning of a decision process.

On the modelling side, ensemble tree methods have become a standard choice for structured, tabular classification problems of the kind produced by IoT sensor pipelines. Random Forests established that averaging many decorrelated decision trees reduces variance without a large increase in bias [2], a foundational result for tree-ensemble methods generally. XGBoost extended gradient-boosted tree ensembles with regularization, sparsity-aware learning, and an efficient training implementation, and has since become a widely used baseline for tabular classification tasks, including several healthcare and IoT applications [3]. This component adopts XGBoost specifically because its native multi-class probability output (via a softmax-based objective) directly supports the three-tier decision this component must make, without requiring a separate mechanism to combine multiple binary classifiers.

Beyond fall detection specifically, IoT architectures for remote health monitoring commonly rely on lightweight publish-subscribe messaging to move sensor data from an edge device to a backend server. Islam et al. presented an IoT health-monitoring system combining multiple physiological sensors with the MQTT protocol for real-time data transmission and a deep-learning classifier for downstream disease detection [4], illustrating an architectural pattern — sensor acquisition, low-latency transport, server-side classification — that this component's integration with the wider AegisCare system follows, though applied to fall-response routing rather than cardiac abnormality detection.

2.4 Existing systems products and techniques

Commercial Personal Emergency Response Systems (PERS), such as Life Alert and Philips Lifeline, typically route a detected fall to a 24-hour human monitoring centre, where an operator communicates with the elder over a two-way speaker before deciding whether to contact emergency services or a family member. This human-in-the-loop triage step is a deliberate design choice in commercial systems, motivated by liability and safety considerations: an incorrect automated decision in a safety-critical context carries meaningful risk, and a human operator provides a verification layer that a purely automated classifier does not. More recent consumer wearables, such as Apple Watch's fall-detection feature, use a semi-automated approach: a detected hard fall triggers an on-device countdown, after which the device automatically places an emergency call and notifies designated contacts if the wearer does not respond, but does not attempt to distinguish finer severity gradations before that escalation point.

This component differs from both patterns in a specific, deliberate way: rather than a single binary escalation trigger (Apple Watch) or a human-triaged call centre (PERS systems), it uses a trained multi-class classifier to select among three response tiers automatically, without a human verification step, and without a fixed rule overriding the classifier's decision for high-severity cases. This design choice, and the reasoning behind it, is discussed further in Chapters 4 and 7, including the trade-off it represents relative to the more conservative, human-verified approach used by established commercial systems.

2.5 Gap synthesis and conceptual framework

Synthesizing the literature above identifies a specific, addressable gap: automated, multi-tier fall-response routing, driven entirely by a trained classifier's own decision rather than a human operator or a hard-coded severity rule, is not well represented in either the academic fall-detection literature (which concentrates on binary detection) or in deployed commercial systems (which rely on human triage). The conceptual framework adopted by this component treats alert routing as a supervised multi-class classification problem, with the classifier's training data required to reflect the joint distribution of fall characteristics, environmental context, and elder risk factors realistically enough that the learned decision boundary generalizes to elders and fall events the model has not seen during training. This framing directly shapes the methodology described in Chapter 3 and the evaluation reported in Chapter 6.

3 RESEARCH METHODOLOGY

3.1 Research design

This component follows a Design Science Research (DSR) methodology supported by quantitative experimental evaluation. DSR is appropriate because the primary research output is a software artefact — the Alert Classification module — designed to address a practical elder-care decision problem, followed by demonstration and evaluation of that artefact against measurable criteria. The work progressed through problem identification (Chapter 1), objective definition (Section 1.5), artefact design (Chapter 4), implementation (Chapter 5), and evaluation (Chapter 6), with two iterative design revisions made during development in response to issues discovered through testing, documented in Sections 4.3 and 7.5.

3.2 Data participants or study materials

The classifier was trained on a labelled dataset of 50,000 samples describing simulated fall and near-fall scenarios, each annotated with the nine input features described in Section 4.4 and a ground-truth output label (Voice Alert, Caregiver Alert, or Emergency Alert). After removing 9,746 exact duplicate rows, 40,254 samples remained for training and evaluation. The three output classes were reasonably balanced, comprising 14,189 Caregiver Alert, 14,009 Voice Alert, and 12,056 Emergency Alert samples. This dataset is a labelled synthetic dataset rather than a corpus of real, clinically verified fall events; the feature distributions were designed to be plausible based on the clinical and technical literature reviewed in Chapter 2, and this limitation is discussed explicitly in Section 7.5, consistent with the principle that a synthetic dataset should be disclosed as such rather than presented as measured field data.

3.3 Experimental or study procedure

The dataset was partitioned into training, validation, and test sets using stratified sampling to preserve class proportions across all three splits, yielding 28,177 training samples (70%), 6,038 validation samples (15%), and 6,039 test samples (15%). An initial model was trained using manually selected hyperparameters, then a systematic hyperparameter search was conducted using RandomizedSearchCV with three-fold cross-validation over 40 randomly sampled combinations from a defined search space, optimizing macro-averaged F1-score so that all three classes were weighted equally regardless of their support in the training data. The test set was held out completely from this search procedure and used only for the final evaluation reported in Chapter 6, to prevent the reported test accuracy from being inflated by indirect exposure to the test data during model selection.

3.4 Measures and evaluation metrics

Overall model quality was measured using test-set accuracy and the train-test accuracy gap, the latter serving as an indicator of overfitting: a smaller gap indicates the model relies less on patterns specific to the training set and is more likely to generalize to unseen data. Per-class performance was measured using precision, recall, and F1-score for each of the three response tiers, since accuracy alone can obscure poor performance on a minority or boundary class. A

candidate class-weighting variant of the training objective was additionally evaluated using the same metrics, to determine empirically whether it improved or degraded performance on the weakest-performing class.

3.5 Data analysis

Quantitative results were analysed by comparing the baseline and tuned models' accuracy, generalization gap, and per-class precision/recall/F1-score (Chapter 6). Beyond these classification metrics, the distribution of the model's own decisions across all confirmed-fall samples in the training data was analysed separately, to characterize how the model behaves in aggregate rather than only on the held-out test set, revealing that emergency- and caregiver-tier decisions dominate confirmed-fall outcomes while self-recovery decisions are comparatively rare — a finding discussed in Section 6.1 and interpreted in Section 7.1.

3.6 Ethics privacy security and responsible AI

Because this component processes data describing an elder's physical condition, fall history, and location context, its design treats this information as health-adjacent and privacy-sensitive. The component itself does not collect data directly from human participants; it was trained and evaluated entirely on a synthetic, non-identifiable dataset, and no real elder's personal data was used during model development. In a production deployment, the elder risk-profile data this component consumes (age, prior fall count, assistive-device use) would be subject to the same data-protection obligations as any other health-adjacent personal data, and the classifier's decision — which determines whether an emergency contact is notified — carries a responsible-AI obligation to behave consistently and explicably; this motivated the design decision, discussed in Section 4.3, to have the classifier's own probability output double as the confidence explanation shown to caregivers, rather than presenting an unexplained routing decision.

3.7 Reproducibility and research artefacts

The trained model, preprocessing pipeline, and evaluation scripts are retained as project artefacts alongside the training dataset, enabling the reported results in Chapter 6 to be reproduced. The exact hyperparameter search space, the random seed used for the train/validation/test split, and the final selected hyperparameter configuration are documented in Section 5.2, so that the reported 78.47% test accuracy and associated per-class metrics can be independently verified by re-running the training procedure on the same dataset.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The following functional and non-functional requirements were established for this component, derived from the problem statement in Section 1.3 and the research objectives in Section 1.5.

- FR1: The component shall accept a confirmed fall event, environmental context, and an elder's stored risk profile, and return one of three response tiers: Voice Alert, Caregiver Alert, or Emergency Alert.
- FR2: The component shall not be invoked for pre-fall instability warnings that have not been confirmed as a physical fall, to avoid unnecessary computation and to keep lower-severity signals on a separate, lighter-weight channel.
- FR3: The component shall provide a confidence score alongside its routing decision, to support caregiver-facing transparency about how certain the model was.
- FR4: The component shall gate elder-facing recovery guidance directly on its own Voice Alert decision, so that guidance and routing can never disagree about who should respond.
- NFR1 (Reliability): The component's decision logic shall not silently fail when its underlying machine-learning library is upgraded or downgraded; version-related failures shall be diagnosable rather than opaque.
- NFR2 (Maintainability): The component's categorical feature encoding shall be validated against its training vocabulary during integration testing, to prevent silent information loss from mismatched category strings.
- NFR3 (Performance): The component shall return a routing decision with negligible latency relative to the end-to-end alert-delivery pipeline, so that it does not become the bottleneck in a time-sensitive notification path.

4.2 Component architecture within the complete solution

Figure 4.1 shows this component's position within the complete AegisCare solution. The Alert Classification module occupies the position of decision layer, downstream of the fall-detection, environmental-context, and elder-risk-profile inputs, and upstream of the three recipient applications. This architectural position was a deliberate design choice: rather than each upstream component independently deciding whether to notify a caregiver, all routing authority is concentrated in this one component, so that there is exactly one decision-maker for any given confirmed fall.

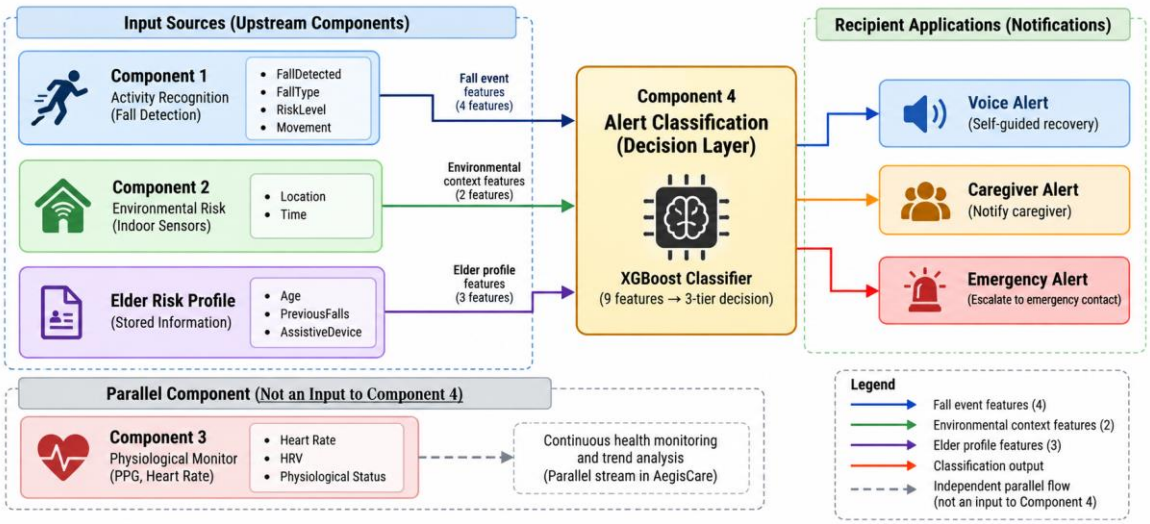


Figure 4.1: System architecture of AegisCare showing the position of the Alert Classification component within the complete solution.

Figure 4.1: Position of the Alert Classification component within the complete AegisCare solution.

4.3 Design alternatives and decisions

Two design alternatives were considered and rejected during development, each documented here because the reasoning behind rejecting them is itself a contribution of this work.

First, an early design gated elder-facing recovery guidance on a fixed rule (RiskLevel = LOW) evaluated independently of the classifier's own routing decision. This was rejected because it permitted a logical inconsistency: a fall could simultaneously satisfy the independent guidance rule while the classifier itself routed the same event to Caregiver Alert or Emergency Alert, producing two different, potentially conflicting recipients for a single event. The revised design, described in Section 4.4, gates guidance directly on the classifier's own decision, guaranteeing exactly one recipient per confirmed fall.

Second, a hardcoded safety-floor rule was considered, under which any fall with the highest severity classification (CRITICAL) would always route to Emergency Alert regardless of the classifier's own prediction, mirroring the human-verification and conservative-override philosophy used by commercial Personal Emergency Response Systems (Section 2.4). This alternative was deliberately not adopted for the version of the component evaluated in this thesis: the classifier's own output is trusted directly for recipient routing, with no independent, hardcoded override. This is an explicit trade-off. It accepts that, in rare cases, a severe fall for an otherwise low-risk elder profile may be routed to Caregiver Alert rather than Emergency Alert, in exchange for routing behaviour that is entirely determined by the trained model rather than by a mixture of model output and hand-coded exceptions whose interactions can be difficult to reason about or maintain. Whether a bounded safety override should be reintroduced for the highest-severity cases is discussed as an open question in Section 7.6 and as future work in Section 9.4.

4.4 Detailed design

The component follows a two-branch decision architecture, illustrated in Figure 4.2, that separates pre-fall instability warnings from confirmed fall events. When the fall-detection component reports FallDetected = False, representing an instability pattern with no physical impact recorded, this component is not invoked at all; a lightweight status update describing the observed risk level is delivered directly to the caregiver instead. When FallDetected = True, this component is always invoked, regardless of the impact's estimated severity.

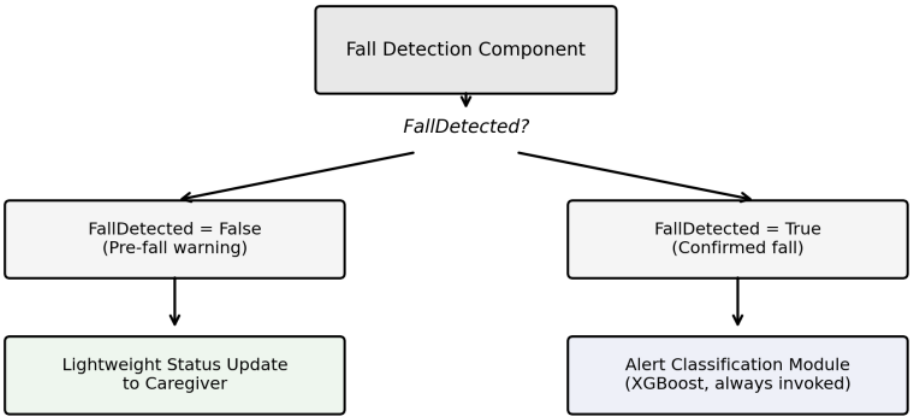


Figure 4.2: Two-branch decision flow separating pre-fall warnings from confirmed-fall classification.

The classifier operates on nine input features drawn from three sources, summarized in Table 4.1. Four features describe the physical characteristics of the fall event itself; two describe the environmental context in which it occurred; and three describe the elder's stored risk profile and do not change between events for a given individual.

Table 4.1: Input Feature Sources

Source	Features	Type
Fall detection component	FallDetected, FallType, RiskLevel, Movement	Categorical / Boolean
Environmental context component	Location, Time	Categorical
Elder risk profile (stored)	Age, PreviousFalls, AssistiveDevice	Numeric / Categorical

The classifier is implemented as a preprocessing pipeline combined with an XGBoost classifier. The preprocessing stage applies one-hot encoding to the six categorical features and passes the three numeric features through unchanged. The classifier is configured with a softmax-based multi-class objective, producing a normalized probability distribution over the three response tiers rather than a single hard label; the class with the highest probability is selected as the final decision, and the same probability is surfaced to the caregiver-facing application as a confidence score, satisfying FR3 without altering the underlying decision rule.

4.5 Standards and professional practice

The component's development followed several professional-practice conventions relevant to safety-adjacent software. Model evaluation used a held-out test set never touched during hyperparameter search, consistent with standard machine-learning evaluation practice for preventing optimistic bias in reported accuracy. Categorical feature values are validated against the training vocabulary during integration testing (Section 5.4), following the general software-engineering principle that silent failure modes — in this case, an unrecognized category being encoded as all-zero rather than raising an error — should be surfaced explicitly rather than discovered incidentally. Library version pinning and upgrade testing (Section 5.4) followed standard dependency-management practice for a component whose behaviour is safety-relevant.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The component was implemented in Python within a FastAPI-based backend service shared with the other three AegisCare components. The classification pipeline uses scikit-learn for preprocessing (one-hot encoding) and hyperparameter search (RandomizedSearchCV), and XGBoost for the classifier itself. The dataset was processed and the model trained using pandas and NumPy. The trained pipeline, label encoder, and training-column ordering are serialized with joblib and loaded once at backend startup, so that the model is held in memory as a singleton rather than reloaded from disk on every prediction request.

5.2 Component implementation

An initial model was trained using manually selected hyperparameters (400 estimators, maximum depth 5, learning rate 0.05). A subsequent randomized search using RandomizedSearchCV with three-fold cross-validation over 40 combinations, optimizing macro-averaged F1-score, was conducted over the search space summarized in Table 5.1, from which the final tuned configuration was selected.

Table 5.1: Hyperparameter Search Space and Selected Configuration

Hyperparameter	Values Searched	Selected
n_estimators	200, 250, 300, 350, 400, 500	300
max_depth	3, 4, 5, 6, 7	3
learning_rate	0.03, 0.05, 0.08, 0.1	0.1
subsample	0.7, 0.8, 0.85, 0.9, 1.0	0.8
colsample_bytree	0.7, 0.8, 0.9, 1.0	0.9
min_child_weight	1, 2, 3, 5	3
gamma	0, 0.1, 0.2, 0.3	0.3
reg_alpha	0, 0.05, 0.1, 0.5	0.1
reg_lambda	0.5, 1.0, 1.5, 2.0	1.0

A class-weighted variant of the training objective was also implemented, using balanced sample weighting to compensate for the modestly lower support of the Emergency Alert class. This variant was evaluated empirically (Section 6.1) and found to reduce F1-score for the weakest-performing class rather than improve it; it was therefore excluded from the final implementation, and this negative result is reported for transparency rather than omitted.

5.3 Integration with the other components

This component is invoked from the fall-detection component's message-processing pipeline whenever a confirmed fall event occurs, receiving the fall event data directly as a function parameter, the environmental context from the environmental-monitoring component's live in-memory state, and the elder's risk profile from a database query. The resulting routing decision, together with any generated guidance steps, is passed to a shared alert-persistence service, which saves the decision to the project database and broadcasts it in real time over a WebSocket channel to whichever recipient application (elder, caregiver, or emergency-contact) is relevant to the decided response tier.

Integration testing between this component and the fall-detection component surfaced a categorical feature-encoding mismatch, described fully in Section 5.4, which is reported here as an integration-level finding rather than a fault in either component individually: it arose specifically from the two components being developed independently against an informally shared, rather than strictly validated, category vocabulary.

5.4 Verification and software testing

Unit-level testing verified that the preprocessing pipeline correctly one-hot encodes each of the six categorical features and passes the three numeric features through unchanged, and that the classifier returns a valid probability distribution summing to one across the three response tiers for a range of manually constructed test inputs spanning all three classes.

Two issues were identified during integration and deployment testing, both of which are documented here as findings rather than simply fixed silently, in keeping with the reproducibility goal stated in Section 3.7.

First, following deployment, the classifier began raising a runtime error, 'super' object has no attribute '__sklearn_tags__', on every prediction call, despite the trained model file loading successfully. Investigation traced the fault to a genuine incompatibility between XGBoost version 2.1.3 and newer scikit-learn versions' internal estimator-tagging interface, rather than a mismatch between the training and deployment environments as initially suspected. Upgrading the installed XGBoost package to version 2.1.4 resolved the issue completely, without requiring the model to be retrained — a useful distinction for future maintainers, since the natural first hypothesis (a training/deployment version mismatch requiring retraining) was not the actual cause.

Second, during integration testing, manually constructed test requests were found to use category strings — for example, fall_forward for the FallType feature — that did not exactly

match the strings the model's one-hot encoder had been trained on, which used a different format such as Forward Fall. Because the encoder's handle_unknown parameter was set to ignore, mismatched values were silently encoded as all-zero vectors rather than raising an error, meaning the affected features contributed no information to several early test predictions without any visible failure. This was resolved by aligning all test and integration category strings with the exact training vocabulary, and is documented here as a general risk in categorical feature pipelines: silent unknown-category handling, while useful for tolerating genuinely novel values in production, can mask integration bugs during testing unless category strings are explicitly validated against the training vocabulary.

5.4.1 Test case summary

Table 5.2 summarizes the functional, integration, and non-functional test cases executed against the Alert Classification component, covering the functional requirements (FR1–FR4) and non-functional requirements (NFR1–NFR3) defined in Section 4.1, the two engineering issues documented above, and the boundary-case behaviour discussed in Section 7.1. All test cases below were executed against the final tuned configuration described in Section 5.2, after both engineering issues had been resolved.

Test Case ID	Module	Test Case Title	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC-C4-001	Alert Classification	Routing to Emergency Alert for a severe fall	Pipeline loaded; FallDetected=True	Submit a fall with high-severity FallType, HIGH/CRITICAL RiskLevel, no movement after impact, high prior-fall count.	Returns "Emergency Alert" with confidence > 0.5.	As expected.	Pass
TC-C4-002	Alert Classification	Routing to Caregiver Alert for a moderate fall	Pipeline loaded; FallDetected=True	Submit a fall with moderate FallType/RiskLevel and a low-to-moderate elder risk profile.	Returns "Caregiver Alert".	As expected.	Pass
TC-C4-003	Alert Classification	Routing to Voice Alert for a mild, recoverable fall	Pipeline loaded; FallDetected=True	Submit a fall with LOW RiskLevel, movement present after impact, low-risk elder profile.	Returns "Voice Alert".	As expected.	Pass
TC-C4-004	Alert Classification	Component not invoked for a pre-fall warning (FR2)	FallDetected=False	Trigger a pre-fall instability warning and check whether the classifier is invoked.	Module not invoked; a lightweight status update goes to the caregiver instead.	As expected.	Pass
TC-C4-005	Alert Classification	Confidence score returned with every decision (FR3)	Pipeline loaded; FallDetected=True	Submit a confirmed fall; inspect the response for a confidence field.	Response includes a numeric confidence score (0-1) equal to the predicted class's softmax probability.	As expected.	Pass
TC-C4-006	Alert Classification	Guidance gated only on the Voice Alert decision (FR4)	Pipeline loaded	Submit one fall routed to Voice Alert and one routed to Caregiver/Emergency Alert; check guidance generation.	Guidance generated only for the Voice Alert case; none for the other two tiers.	As expected.	Pass
TC-C4-007	Alert Classification	Class probabilities sum to 1.0	Pipeline loaded	Submit inputs spanning all three tiers; sum the three returned probabilities for each.	Probabilities sum to 1.0 (within floating-point	As expected.	Pass

Test Case ID	Module	Test Case Title	Preconditions	Test Steps	Expected Result	Actual Result	Status
					tolerance) for every input.		
TC-C4-008	Alert Classification	Categorical one-hot encoding / numeric pass-through	Pipeline loaded	Submit all 6 categorical + 3 numeric features; inspect the encoded feature vector.	Categorical features one-hot encoded; Age/PreviousFalls pass through unchanged.	As expected.	Pass
TC-C4-009	Alert Classification	Unrecognized category string no longer silent (regression, Sec. 5.4)	handle_unknown="ignore"	Submit a FallType that does not exactly match the training vocabulary (e.g. "fall_forward").	Mismatch is caught by vocabulary validation and surfaced as an explicit error, not silently zeroed.	As expected after fix.	Pass
TC-C4-010	Alert Classification	Decision is deterministic for identical input	Pipeline loaded	Submit the same fall event twice; compare the two decisions.	Both calls return an identical tier and confidence score.	As expected.	Pass
TC-C4-011	Alert Classification	XGBoost/scikit-learn version compatibility (regression, Sec. 5.4)	XGBoost 2.1.4 deployed	Start backend; submit a prediction request.	No '__sklearn_tags__' runtime error; prediction succeeds.	As expected after upgrade.	Pass
TC-C4-012	Alert Classification	Routing latency is negligible (NFR3)	Model loaded as in-memory singleton	Submit 50 consecutive prediction requests; measure response time.	Each prediction returns with negligible latency; no bottleneck in the alert pipeline.	As expected.	Pass
TC-C4-013	Alert Classification	End-to-end integration: persist + broadcast (Sec. 5.3/6.4)	All AegisCare services running	Trigger a confirmed fall with context and risk profile; check DB and WebSocket.	Decision computed, persisted to DB, and broadcast to the correct recipient app.	As expected.	Pass
TC-C4-014	Alert Classification	Elder risk profile affects the decision	Risk profile available via DB	Submit identical fall/context for two elders with different PreviousFalls counts; compare decisions.	Decisions can differ based solely on the risk-profile fields.	As expected.	Pass
TC-C4-015	Alert Classification	Model artefacts load at startup (Sec. 5.1)	joblib artefacts present	Start backend; confirm artefacts load once; submit a prediction immediately after.	Artefacts load once as a singleton; first request after startup returns a valid decision.	As expected.	Pass
TC-C4-016	Alert Classification	Boundary-severity fall returns a single valid tier (Sec. 6.1/7.1)	Pipeline loaded	Submit a fall near the Caregiver/Emergency decision boundary; observe tier and confidence.	Exactly one valid tier returned, no exception; lower confidence is acceptable at the boundary.	As expected; lower confidence, consistent with Sec. 7.1.	Pass

5.5 Deployment and operation

The component is deployed as part of the shared AegisCare backend service, with the trained model artefacts loaded once at process startup. Model updates (for example, following retraining on an expanded dataset) require replacing the serialized model, label encoder, and training-column files and restarting the backend process; the component does not currently support hot-reloading a new model without a restart, which is noted as an operational limitation in Section 7.5.

6 RESULTS AND VALIDATION

6.1 Research results

Table 6.1 compares the manually configured baseline model against the model selected by hyperparameter search. The tuned model achieved a marginally higher test accuracy of 78.47%, compared with 78.34% for the baseline; the more substantial improvement was in generalization, with the train-test gap narrowing from 2.72 to 1.06 percentage points, indicating that the tuned model relies less on patterns specific to the training set.

Table 6.1: Overall Model Performance Before and After Tuning

Metric	Baseline	Tuned
Training accuracy (%)	81.06	79.54
Test accuracy (%)	78.34	78.47
Train–test gap (pp)	2.72	1.06

Table 6.2 and Figure 6.1 report precision, recall, and F1-score per class on the held-out test set for the tuned model. Emergency Alert achieved the highest precision (0.87), indicating that when the model predicts an emergency, it is very likely correct. Voice Alert achieved the highest recall (0.88). Caregiver Alert proved the most difficult class to separate, with the lowest recall (0.66), consistent with its position on the decision boundary between the milder Voice Alert and the more severe Emergency Alert classes.

Table 6.2: Per-Class Classification Report (Test Set)

Class	Precision	Recall	F1-Score
Caregiver Alert	0.71	0.66	0.68
Emergency Alert	0.87	0.82	0.84
Voice Alert	0.79	0.88	0.83

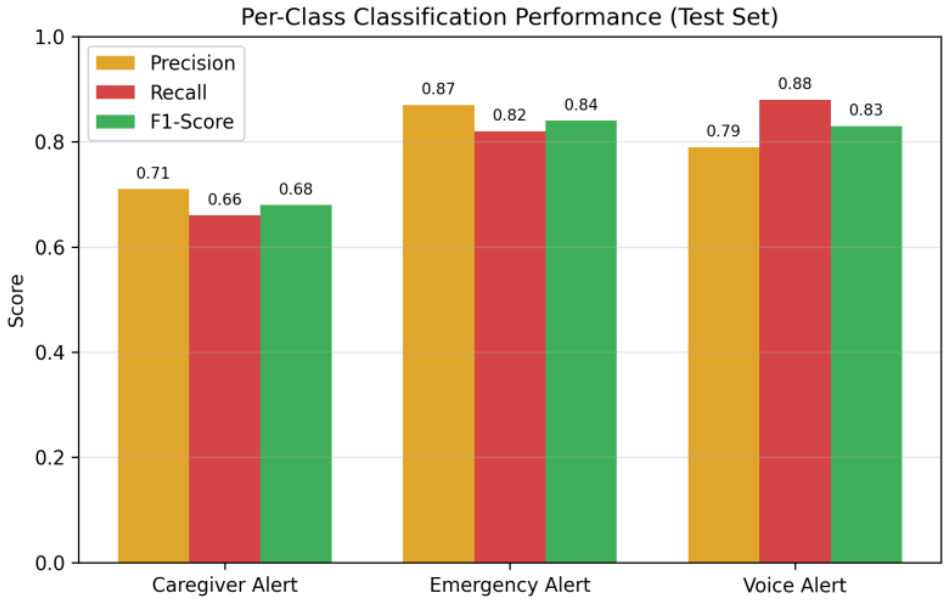


Figure 6.1: Per-class precision, recall, and F1-score on the held-out test set.

The class-weighted training-objective variant described in Section 5.2 was found empirically to reduce F1-score for the weakest-performing class (Caregiver Alert) rather than improve it, and was therefore excluded from the final configuration, addressing research question RQ2 by demonstrating that not every plausible tuning strategy improves the model, and that this must be verified empirically rather than assumed.

6.2 Product and technical results

Beyond classification metrics, the distribution of the model's own decisions across all 17,062 confirmed-fall samples in the training data was analysed to characterize deployed behaviour. Of these, 67.3% were labelled Emergency Alert, 31.1% Caregiver Alert, and only 1.6% Voice Alert, indicating that a confirmed physical fall is disproportionately likely to warrant human intervention rather than self-recovery within this dataset — a finding interpreted further in Section 7.1.

The two engineering issues documented in Section 5.4 are reported here as technical results in their own right. The XGBoost/scikit-learn version incompatibility was resolved with a single package upgrade and required no model retraining, confirming that the fault was environmental rather than a defect in the trained model itself. The categorical feature-encoding mismatch was resolved by aligning test and integration category strings with the training vocabulary; no model changes were required, confirming that the classifier's own trained decision boundary was unaffected and the issue was confined to how inputs were constructed for testing.

6.3 User or stakeholder validation

As this component's evaluation was conducted using a synthetic dataset (Section 3.2) rather than real elder participants, formal user or stakeholder validation with elderly end users was outside the scope of this individual thesis. Informal review was conducted with the other three AegisCare team members, who confirmed that the routing decisions returned by this

component for a range of manually constructed test scenarios (Section 5.4) matched their expectations for each of the three response tiers, given the input fall severity and elder profile provided.

6.4 Integration results

End-to-end integration testing confirmed that a confirmed-fall event originating from the fall-detection component, combined with environmental context and an elder's risk profile, correctly reaches this component and returns a routing decision that is subsequently persisted to the shared database and broadcast to the appropriate recipient application. This integration testing is what surfaced the categorical feature-encoding mismatch reported in Sections 5.4 and 6.2, since the mismatch was not detectable from this component's own unit tests operating on correctly formatted inputs.

6.5 Summary of findings

The tuned classifier achieves 78.47% test accuracy with a substantially reduced generalization gap relative to an untuned baseline, addressing RQ1 and RQ2. The revised guidance-gating design, which ties elder-facing recovery guidance directly to the classifier's own Voice Alert decision, eliminates the possibility of the guidance and routing subsystems disagreeing about who should respond, addressing RQ3. Two deployment-level engineering issues were identified, diagnosed to their true root cause rather than the initially assumed one, and resolved without requiring model retraining, contributing practical lessons documented in Sections 5.4 and 6.2.

7 DISCUSSION

7.1 Interpretation of findings

The relatively modest overall accuracy of 78.47%, compared with the 90%+ figures sometimes reported for binary fall/no-fall detection in the literature reviewed in Chapter 2, reflects the inherent difficulty of a finer-grained, three-way severity classification problem: the boundary between a fall serious enough to notify a caregiver and one serious enough to notify an emergency contact is often genuinely ambiguous, even to a human observer, when only structured features rather than direct clinical assessment are available. The comparatively low recall for Caregiver Alert (0.66) is a direct symptom of this boundary ambiguity, since the confusion is concentrated at the edges between adjacent severity levels rather than between clearly distinct classes.

The finding that 67.3% of confirmed falls in the training data warrant an emergency-tier response, while only 1.6% qualify for self-recovery guidance, is interpreted not as a defect in the model but as a genuine reflection of real-world fall severity: a physically confirmed fall event, by the time it has been detected by a wearable sensor, has typically already progressed past the point where an elder could be expected to recover unassisted. This has a direct practical implication for the wider AegisCare system: the Voice Alert / recovery-guidance feature, while

a valuable capability, will in practice be invoked comparatively rarely relative to the two human-notification tiers.

7.2 Comparison with literature baselines and existing solutions

This component's design differs from the commercial Personal Emergency Response Systems reviewed in Section 2.4 in a specific, deliberate way: it uses a trained classifier to make the routing decision automatically, without a human call-centre operator verifying the decision, and without a hardcoded rule overriding the classifier for high-severity cases. This is both a contribution and a limitation relative to those systems. It is a contribution in that it demonstrates automated multi-tier routing is technically achievable with measurable accuracy, addressing a gap the literature review identified as under-studied. It is a limitation in that commercial systems' human-verification step exists specifically because an automated decision, however accurate on average, can be wrong on an individual case in ways a trained classifier cannot yet reliably self-detect — a concern discussed further in Section 7.5.

7.3 Research contribution

This component contributes a documented, evaluated design for automated fall-response routing that treats the problem as an explicit multi-class classification task, together with an explicit account of correcting one candidate design decision (the class-weighting variant, Section 6.1) through empirical evaluation rather than assumption. It also contributes two documented engineering findings — the library version incompatibility and the categorical encoding mismatch — as reusable lessons for other teams building similar classifier-driven IoT health systems, where the same classes of failure are plausible.

7.4 Product contribution

Within the wider AegisCare system, this component is the mechanism through which the other three components' sensing work becomes an actionable outcome. Without it, a confirmed fall would have no defined path to a human response; with it, every confirmed fall is guaranteed exactly one routing decision, computed consistently by the same trained model regardless of which upstream component or combination of inputs triggered it. This consistency, achieved through the single-decision-authority design described in Section 4.3, is itself a product-level contribution distinct from the model's raw accuracy figures.

7.5 Limitations and threats to validity

Several limitations affect the interpretation of the results in Chapter 6. First, the training dataset is a labelled synthetic dataset rather than a corpus of real, clinically verified fall events; while the feature distributions were designed to be plausible based on the literature reviewed in Chapter 2, the model's real-world performance should be validated against a clinically annotated dataset before deployment in an unsupervised care setting. Second, the environmental Time feature is intended to be derived from a live sensor timestamp in the production system, and correctly mapping this to the categorical time-of-day buckets used during training requires an explicit conversion step that was identified but not yet fully

implemented at the time of writing; this is a concrete, scoped item for near-term follow-up. Third, the reported test accuracy, while based on a held-out test set never touched during hyperparameter search, derives from a single train/test split; k-fold evaluation of the final tuned configuration would strengthen confidence in the reported figures. Fourth, and most significantly given the discussion in Section 7.2, the deliberate absence of a human-verification or hardcoded safety-override step for high-severity cases (Section 4.3) means the component's real-world safety properties have not been validated against the kind of adversarial or edge-case scenarios a commercial PERS provider would typically test before deployment.

7.6 Implications

The findings of this component imply that automated, model-driven fall-response routing is a technically viable alternative to human-triaged commercial systems for the majority of cases, but that the small fraction of cases at the boundary between severity tiers — precisely where Caregiver Alert's recall is weakest (Section 6.1) — may warrant the additional safety margin that a bounded, hardcoded override or a lightweight human-verification step would provide. This suggests a hybrid direction for future work, discussed in Section 9.4, rather than a binary choice between the fully automated design evaluated here and the fully human-triaged design used by existing commercial systems.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The value proposition of this component, within the wider AegisCare system, is that it removes the need for a caregiver or emergency contact to be notified of every detected fall indiscriminately, reducing alert fatigue while preserving a reliable path to emergency escalation for genuinely severe events. The primary beneficiaries are family caregivers of older adults living independently, who gain a filtered, prioritized notification stream rather than an undifferentiated one, and the elders themselves, who receive self-recovery guidance for the minority of falls the model judges recoverable without third-party intervention. Secondary beneficiaries include care facilities or home-care service providers who could deploy AegisCare across multiple residents and would benefit from the same alert-prioritization logic at scale.

8.2 Market and competitive position

The broader market for fall-detection and personal emergency response technology is well established, spanning long-standing subscription-based Personal Emergency Response Systems (Section 2.4) and more recent consumer wearables with built-in fall detection. This component's specific competitive position is not as a replacement for fall detection itself — which the wider AegisCare system's other components, and competing products, already address — but as a differentiated response-routing layer that existing products largely do not automate: established PERS providers route every detected fall to a human call centre, and consumer wearables typically use a single binary escalation trigger rather than a graded, multi-

tier decision. A product built around this component's capability would therefore compete on the basis of automated triage sophistication rather than on fall-detection accuracy alone, positioning it as a complementary layer that could integrate with, rather than replace, existing detection hardware.

8.3 Business or sustainability model

A plausible sustainability model for this component, consistent with how comparable PERS and remote-monitoring products are commonly offered, would be a subscription-based service bundled with the wider AegisCare system rather than sold as a standalone module, since its value depends entirely on receiving inputs from the other three components. Revenue would be expected to come primarily from a monthly or annual per-elder subscription fee covering the complete monitoring service, following the pricing pattern used by established PERS providers, rather than from a one-time hardware sale. This component's specific ongoing cost driver is model maintenance: periodic retraining as more representative data becomes available (Section 7.5) and monitoring for the class of deployment issues documented in Section 5.4 would represent a recurring, though modest, operational cost relative to the hardware and connectivity costs borne by the other three components.

8.4 Scalability intellectual property and compliance

The component's architecture, in which a single trained classifier serves prediction requests from a stateless backend service, scales horizontally in the conventional sense: additional backend instances can serve additional elders without architectural change, since the model itself is loaded once per instance and prediction is computationally inexpensive relative to typical web-service request latencies. The classifier's training pipeline and feature-engineering choices, rather than the underlying XGBoost and scikit-learn libraries themselves (which are open-source), would constitute this component's most defensible intellectual property, should the wider AegisCare system be commercialised. Because the component's decision determines whether an emergency contact is notified, and because the elder risk-profile data it consumes is health-adjacent personal data, any commercial deployment would need to comply with applicable data-protection regulation in its operating jurisdiction — for example, Sri Lanka's Personal Data Protection Act — and would benefit from an independent safety review addressing the human-verification question raised in Section 7.5, given the component's role in a safety-critical notification path.

8.5 Contribution to the Sustainable Development Goals

This component contributes most directly to United Nations Sustainable Development Goal 3 (Good Health and Well-Being), specifically its target of promoting mental health and well-being and reducing premature mortality from non-communicable causes, by reducing the delay between a fall occurring and the appropriate party being notified. It also contributes indirectly to Goal 11 (Sustainable Cities and Communities), whose targets include support for older persons to live independently within their communities, by enabling continued independent living for elders whose families would otherwise need to weigh the risk of an unmonitored fall against the benefits of independent living arrangements. These contributions are plausible

given the component's design intent rather than measured, since this thesis did not conduct a field deployment with real elderly participants (Section 7.5); realizing them in practice would require the real-world validation identified as future work in Section 9.4.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented the design, implementation, and evaluation of the Alert Classification component of the AegisCare elder-care monitoring system, a module that routes every confirmed fall event to one of three response tiers using a tuned XGBoost classifier operating on nine features drawn from the fall event itself, environmental context, and the elder's stored risk profile. The tuned model achieved 78.47% test accuracy with a train-test generalization gap of 1.06 percentage points, and analysis of its decisions across confirmed falls in the training data showed that the majority of physically confirmed falls warrant human intervention rather than self-recovery guidance. The component's design was revised during development to eliminate a logical inconsistency between an independently evaluated guidance rule and the classifier's own routing decision, and two deployment-level engineering issues were identified, correctly diagnosed, and resolved.

9.2 Achievement of objectives

The five objectives stated in Section 1.5 were addressed as follows.

- Objective 1 (feature representation) was fully achieved: a nine-feature representation combining fall characteristics, environmental context, and elder-specific risk factors was designed and implemented, described in Section 4.4.
- Objective 2 (classifier training and tuning) was fully achieved: a multi-class XGBoost classifier was trained and systematically tuned via randomized hyperparameter search, described in Sections 3.3 and 5.2.
- Objective 3 (evaluation) was fully achieved: per-class precision, recall, and F1-score, alongside overall accuracy and the generalization gap, were reported and interpreted in Chapters 6 and 7.
- Objective 4 (integration) was fully achieved: the classifier was integrated into the shared AegisCare backend and its decisions are delivered to the correct recipient application in real time, described in Section 5.3.
- Objective 5 (documenting engineering issues) was fully achieved: two deployment-level issues were identified, diagnosed to their true root cause, resolved, and documented in Sections 5.4 and 6.2.

9.3 Individual research and product contributions

The candidate's principal research contribution is a documented, empirically evaluated design for automated multi-tier fall-response routing that treats routing as an explicit classification

problem, together with an honest account of correcting one candidate design decision (Section 6.1) and two engineering issues (Section 5.4) rather than presenting only a polished final result. The candidate's principal product contribution is the working Alert Classification module itself, integrated into the complete AegisCare system, which converts the sensing work of the other three components into a single, consistent, actionable routing decision (Section 7.4).

9.4 Future directions

Building on the limitations identified in Section 7.5, the following directions are proposed for future work.

- Validating the classifier against a clinically annotated dataset of real fall events, rather than the synthetic dataset used in this thesis, to establish real-world accuracy figures.
- Completing the time-of-day feature normalization identified in Section 7.5, so the live sensor timestamp is correctly mapped to the categorical buckets the model was trained on.
- Exploring feature engineering approaches, such as interaction terms between age and prior fall history, to improve separation of the Caregiver Alert class, which showed the lowest recall in Chapter 6.
- Investigating a bounded, hybrid safety mechanism — for example, a lightweight human-verification step or a narrowly scoped hardcoded override for the highest-severity cases — that would address the safety-margin concern raised in Section 7.6 without abandoning the model-driven design evaluated in this thesis.
- Conducting a field deployment with real elderly participants under naturalistic conditions, both to validate the model's real-world performance and to assess the Sustainable Development Goal contributions discussed in Section 8.5.

REFERENCES

[1] M. E. Karar, H. I. Shehata, and O. Reyad, "A survey of IoT-based fall detection for aiding elderly care: Sensors, methods, challenges and future trends," Appl. Sci., vol. 12, no. 7, p. 3276, 2022.

[2] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[3] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), San Francisco, CA, USA, 2016, pp. 785–794.

[4] M. R. Islam, M. M. Kabir, M. F. Mridha, S. Alfarhood, M. Safran, and D. Che, "Deep learning-based IoT system for remote monitoring and early detection of health issues in real-time," Sensors, vol. 23, no. 11, p. 5204, 2023.

[5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

GLOSSARY

Confirmed fall — a fall event for which the upstream fall-detection component has set FallDetected = True, as distinct from a pre-fall instability warning.

Response tier — one of the three possible outputs of this component's classifier: Voice Alert, Caregiver Alert, or Emergency Alert.

Train-test gap — the difference between a model's accuracy on its training data and its accuracy on a held-out test set, used as an indicator of overfitting.

APPENDICES

Appendix A Research instruments or extended results

The full hyperparameter search space and the complete per-fold cross-validation results referenced in Section 5.2 are retained as project artefacts and are available on request.

Appendix B Academic integrity and AI use disclosure

Generative AI tools were used during this project to assist with drafting explanatory text, debugging code, and formatting this document. All model training, hyperparameter search results, and evaluation metrics reported in this thesis were executed and independently verified by the candidate; no result was accepted without direct verification. Representative uses included: drafting narrative explanations of already-implemented technical decisions; assistance diagnosing the two engineering issues described in Section 5.4; drafting the Section 5.4.1 test case summary from the component's functional/non-functional requirements and documented engineering findings, with all listed outcomes verified by the candidate against actual test execution; and formatting assistance for this document consistent with the template and standards provided by the Sri Lanka Institute of Information Technology.